Kolhapur Institute of Technology's
College of Engineering Kolhapur
(Empowered Autonomous)

R. S. No. 199B/1-3 Gokul Shirgaon, Kolhapur - 416 234

Department of Computer Science & Engineering

(Artificial Intelligence & Machine Learning / Data Science

Project Phase – I Review – I Presentation

DPI Engine: Real-Time Network Traffic Classification & Rule-Based Filtering

Vishal Desai **2425010086**
Yash Sankpal **2324000713**
Aftabalam Makandar **2425010026**
Pratik Mahesh Chopade **2324001102**
Pranav Dhakane **2324000697**

Under the guidance of
Ms Vajreswari Kulal
Department of Computer Science & Engineering (AIML)
KIT's College of Engineering Kolhapur

Contents

1. Introduction & Background
2. Literature Survey
3. Research Gaps Identified
4. Problem Statement
5. Objectives of Proposed Research Work
6. Architecture of Proposed Research Work
7. Proposed Methodology
8. Work-flow of Proposed Work

Introduction

- **Deep Packet Inspection (DPI)** is a network-monitoring technique that examines packet payloads, not just headers, unlike traditional firewalls that only check source/destination addresses.
- Every packet is layered: Ethernet → IP → TCP/UDP → Application data — like a nested set of envelopes.
- A connection is uniquely identified by its five-tuple: source/destination IP, source/destination port, and protocol.
- Even encrypted HTTPS traffic leaks the destination domain in plaintext, via the TLS Client Hello's Server Name Indication (SNI) field.

Literature Survey

#	Representative Works	Key Ideas / Approach	Dataset Used	Strengths	Performance Metrics	Research Gaps / Limitations
1	Azab et al. (2022) — Network Traffic Classification: Techniques & Challenges	Surveys port-based, DPI-based & ML-based classification	Multiple public traffic datasets (survey)	1. Broad comparison of approaches & trade-offs	Qualitative comparison of accuracy vs. overhead	No single approach handles encrypted traffic at scale
2	Rezaei & Liu (2019) — Deep Learning for Encrypted Traffic Classification, IEEE	Reviews deep-learning models for encrypted traffic	Encrypted traffic captures (survey of DL datasets)	1. Classifies traffic without decrypting payloads	Notes declining DPI accuracy as encryption grows	High cost; motivates lighter, rule-based alternatives

[1] Azab et al. (2022): https://www.sciencedirect.com/science/article/pii/S2352864822001845
[2] Rezaei & Liu (2019): https://arxiv.org/abs/1810.07906

Research Gaps Identified

- **Header-only inspection is insufficient —** Traditional, header-only firewalls inspect only IP/port headers, so they cannot tell apart applications that share the same port (e.g. video streaming vs. a video call, both “port 443, TCP”).
- **Payload encryption defeats classic DPI —** Classic string-matching DPI cannot read TLS/HTTPS payloads, so it fails as encrypted traffic becomes the norm.
- **Little flow-aware blocking —** Existing lightweight approaches rarely track connections as five-tuple flows, leading to blocking decisions that are either too late or drop an entire connection incorrectly.
- **Limited scalability —** Single-threaded inspectors cannot keep up with high packet rates on real, large-scale network captures.
- **Gap this project addresses —** SNI-based, flow-tracked classification (as in nDPI/ntop) remains under-explored as a lightweight, open-source, multi-threaded solution for real-time use.

Problem Statement

Most modern internet traffic is encrypted end-to-end (HTTPS/TLS), yet networks still need to identify, classify, and control application traffic — for security, fair usage, parental control, and policy enforcement.

- **Header-only firewalls fall short —** Traditional firewalls inspect only IP/port headers, so they cannot tell YouTube traffic from a video call — both simply look like "port 443, TCP."
- **Payload is encrypted —** TLS/HTTPS hides the actual content, so classic string-matching DPI on payload bytes cannot identify the application being used.
- **No flow-aware blocking —** An app can only be identified after its first few packets — a naive system either blocks too late or drops an entire connection incorrectly.
- **Scale under load —** A single-threaded inspector cannot keep up with high packet rates on real, large-scale network captures.

Objectives of Proposed Research Work

1. **Parse & Identify —** design a DPI engine that parses Ethernet/IP/TCP/UDP headers and extracts the TLS SNI / HTTP Host field to identify the requesting application.
2. **Rule-Based Filtering —** implement a flow-based rule engine that forwards or drops traffic by source IP, application type, or domain — tracked per five-tuple flow.
3. **Scale with Concurrency —** build a multi-threaded, producer–consumer pipeline (load-balancer + fast-path threads) so large packet captures process efficiently.
4. **Report & Visualize —** generate a clear processing report — protocol mix, application breakdown, forwarded vs. dropped counts — from any input PCAP.

Architecture Proposed Research Work

- **Packet Capture Layer —** reads packets from a PCAP file or a live interface (libpcap-based).
- **Parsing & SNI-Extraction Layer —** decodes the Ethernet → IP → TCP/UDP header stack and extracts the TLS Client Hello's SNI (or HTTP Host) field.
- **Flow-Tracking Table —** maintains a table of active connections keyed by the five-tuple (source/destination IP, source/destination port, protocol).
- **Rule Engine —** matches each tracked flow against configurable rules (source IP, application type, or domain) and decides forward or drop.
- **Multi-threaded Producer–Consumer Pipeline —** a load-balancer thread distributes packets across fast-path worker threads for concurrent, high-throughput processing.
- **Reporting Module —** aggregates protocol mix, application breakdown, and forwarded vs. dropped counts into a processing report.

Proposed Methodology

1. **Capture —** read packets from an input PCAP file or live network interface.
2. **Parse —** decode Ethernet/IP/TCP/UDP headers for each packet.
3. **Identify —** extract the TLS SNI / HTTP Host field to determine the requested application or domain.
4. **Track —** look up or create a flow entry keyed by the five-tuple for the connection.
5. **Filter —** match the flow against the rule engine (source IP, application type, or domain) to decide forward or drop.
6. **Report —** update per-flow statistics and generate a processing report of protocol mix and forwarded/dropped counts.

Work-flow of Proposed Work

- **Start** → Input PCAP / live traffic capture
- **Packet Parsing** → Ethernet / IP / TCP–UDP header decoding
- **Application Identification** → TLS Client Hello SNI / HTTP Host field read
- **Flow-Table Lookup** → lookup/create entry keyed by the five-tuple
- **Rule Matching** → compare flow against source IP / app-type / domain rules
- **Forward / Drop Decision** → forward or drop the packet based on the matched rule
- **Stats Update** → update per-flow counters and protocol/application statistics
- **End —** processing report (protocol mix, forwarded vs. dropped counts)